Central Collection, CSV Logs, And Data Meaning

This document explains what the central computer receives, what gets written to CSV, how timestamps work, what the values mean, and how the processing switches affect the data.

Table Of Contents

1. Quick Summary

There are two different CSV capture paths:

CSV path	Where it runs	Best for
Central OSC collector CSV	central computer	raw archive of every OSC packet from many Pis and microphones
Pi-local analysis CSV	each Raspberry Pi	regular time-grid analysis table for one microphone

The central computer does not open microphones and does not run VAD, openSMILE, or emotion2vec. Those models run on each Pi. The central computer receives OSC messages, displays them in the browser, sends control commands, and can archive the OSC stream.

For real sessions, VAD should normally be active. Without VAD, the system treats the gate as open and may process silence, room noise, handling sounds, or electrical noise as if it were speech.

2. System Roles

Each Raspberry Pi runs one `strip_monitor.py` process per microphone. That Pi process is responsible for:

- opening the local Linux audio input named in `config_mic1.yaml` or `config_mic2.yaml`
- resampling microphone audio to the project sample rate, normally 16 kHz
- running VAD, openSMILE prosody, and emotion inference when those stages are enabled
- sending OSC telemetry to the central computer
- optionally writing a Pi-local CSV log under `output/`

The central computer can run:

- `receiver/bridge.js`, launched by `./run_web.sh`, for the browser GUI and remote control buttons
- `osc_collector.py`, for central CSV capture of the OSC stream

The central computer identifies streams by logical `device_id`, not by Linux microphone names. A device id is built on the Pi as `<pi_id>-<mic_id>`, for example `5-1` or `5-2`.

3. Discovery And OSC Routing

Every Pi process periodically broadcasts:

```
/hello <device_id> <pi_id> <mic_id> <hostname> <ctrl_port> <version>
```

This lets the central bridge discover Pis and route control commands back to the correct process.

Analysis data is namespaced under:

```
/dev/<device_id>/...
```

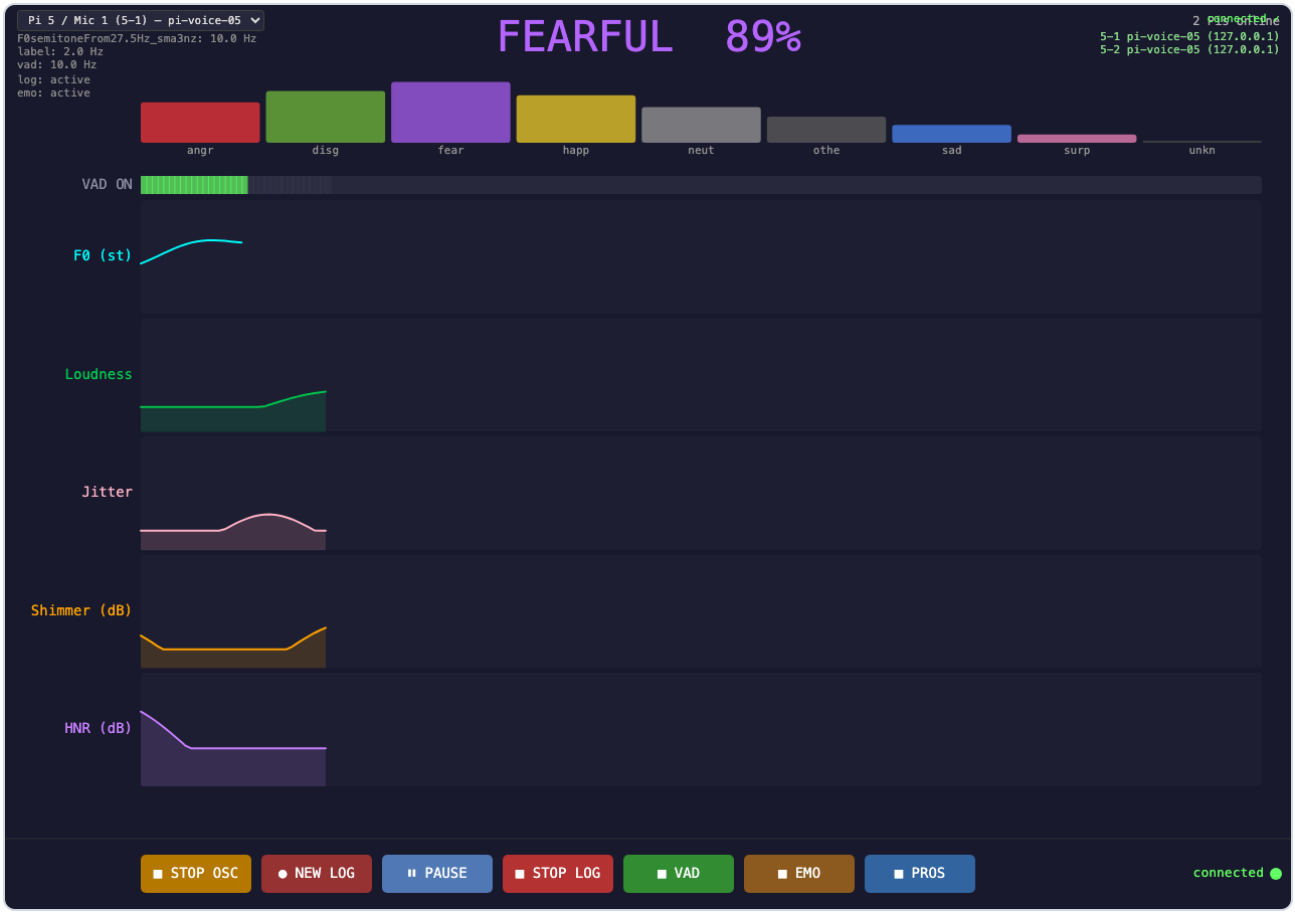
For example:

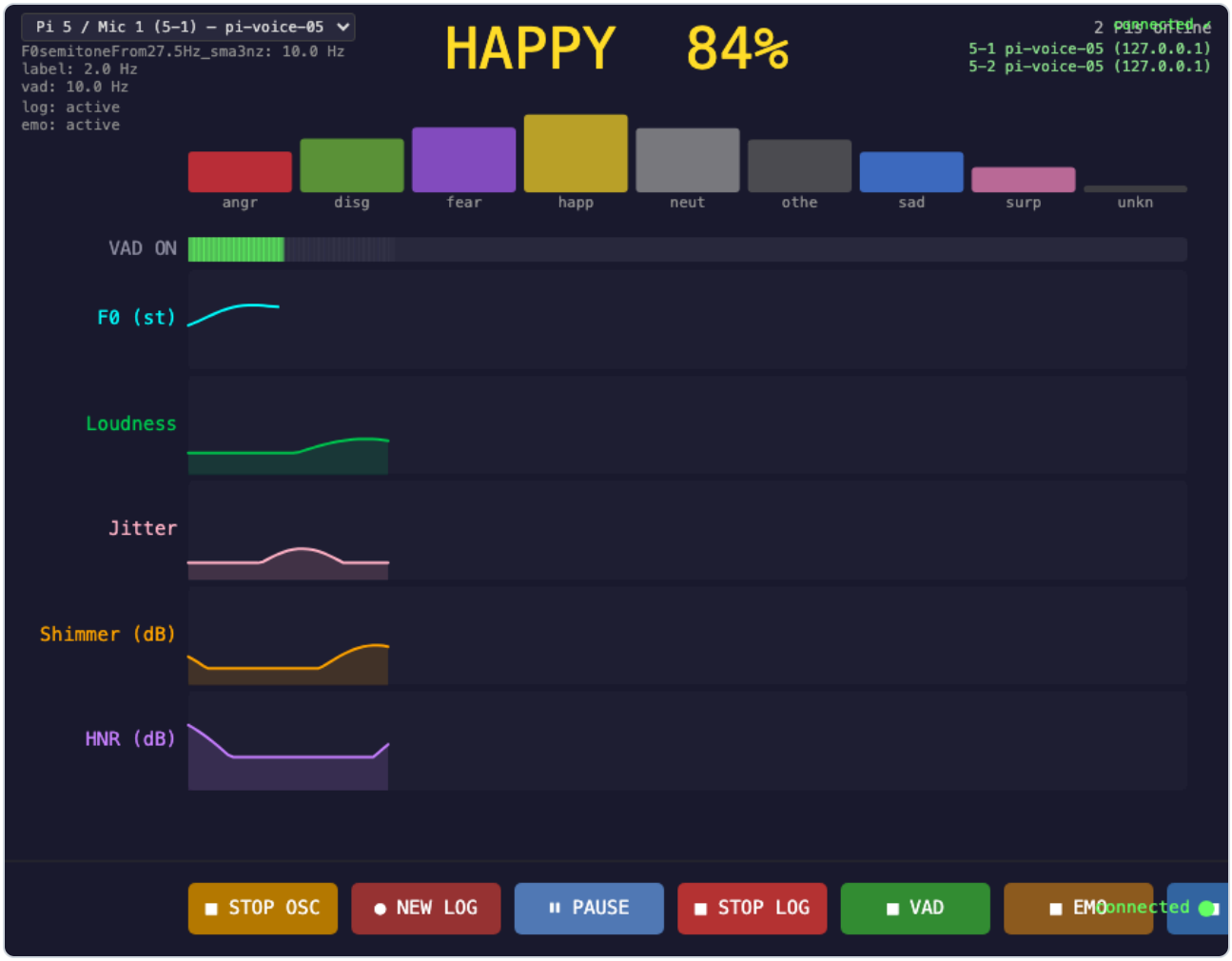
```
/dev/5-1/speech/vad
/dev/5-1/speech/F0semitoneFrom27.5Hz_sma3nz
/dev/5-1/speech/emo/label
/dev/5-1/stats/self
```

This namespace is what lets many Pis and many microphones share one central OSC port.

3.1 Browser Receiver Screenshots

The browser receiver is the central visual control surface. These screenshots use simulated OSC data, but the layout is the same when real Pis are broadcasting.


Pi 5 / Mic 1 (5-1) — pi-voice-05
F0semitoneFrom27.5Hz_sma3nz: 10.0 Hz
label: 2.0 Hz
vad: 10.0 Hz
log: active
emo: active
FEARFUL 89%
2 connected
5-1 pi-voice-05 (127.0.0.1)
5-2 pi-voice-05 (127.0.0.1)
angr
disg
fear
happ
neut
othe
sad
surp
unkn
VAD ON
F0 (st)
Loudness
Jitter
Shimmer (dB)
HNR (dB)
STOP OSC
NEW LOG
PAUSE
STOP LOG
VAD
EMO
PROS
connected



4. CSV Outputs

4.1 Central OSC Collector CSV

Run the central collector on the central computer:

```
source venv/bin/activate
python osc_collector.py --bind 0.0.0.0 --port 9000 --out output/multi
```

`osc_collector.py` writes one CSV file per `device_id`, opened when the first `/dev/<device_id>/...` packet arrives:

```
output/multi/<YYYYMMDD_HHMMSS>_<device_id>.csv
```

The central collector CSV uses a long schema: one row per OSC message.

Column	Meaning
`recv_ts_iso`	UTC receive time on the central computer, ISO formatted.
`recv_ts_unix`	UTC receive time on the central computer, Unix seconds with decimals.

Column	Meaning
`sender_ip`	IP address that sent the OSC packet.
`device_id`	Logical stream id, for example `5-1`.
`address`	Full OSC address, for example `/dev/5-1/speech/vad`.
`args_json`	OSC arguments serialized as JSON.

This format deliberately does not create one column per feature. It accepts any current or future OSC topic without changing the schema. For analysis, pivot this long table into a wide table later if needed.

Important timestamp detail: `recv_ts_iso` and `recv_ts_unix` are central-computer receive times, not the original audio sample times on the Pi. They are good for ordering packets and approximate wall-clock alignment, but they include network and scheduling jitter.

4.2 Pi-Local Analysis CSV

The Pi can also write a local CSV when logging is enabled from the browser, from `/ctrl/log_start`, or by setting `log_active: true` in the config. These files are written under the Pi's configured `output_dir`, normally:

```
output/track_<YYYYMMDD_HHMMSS>.csv
```

The Pi-local CSV is a wide analysis table. One row is written on the logger clock, controlled by `log_interval` in the config. The default is:

```yaml
log_interval: 0.25
```

That means the Pi-local CSV is normally written at 4 rows per second.

The Pi-local CSV columns are:

Column	Meaning
`session_start_unix_ms`	Unix timestamp in milliseconds for the logging session start. Repeated on every row.
`session_start_iso`	ISO timestamp for the logging session start. Repeated on every row.
`timestamp_unix_ms`	Unix timestamp in milliseconds for this row. This includes the calendar date.
`timestamp_iso`	ISO timestamp for this row. This includes the calendar date.
`time_ms`	Elapsed recorded time in milliseconds since the current log session started. This pauses/resumes with logging.
`vad`	Tri-state VAD gate: `-1` = VAD off/no data, `0` = silence, `1` = speech.
prosody columns	Latest available openSMILE low-level descriptor values. Blank if unavailable.
`emo_label`	Current top emotion label, blank when emotion processing is off.
`emo_confidence`	Confidence of `emo_label`, blank when emotion processing is off.
emotion score columns	One score per emotion dimension: `angry`, `disgusted`, `fearful`, `happy`, `neutral`, `other`, `sad`, `surprised`, `unknown`.

The row-level `timestamp_unix_ms` and `timestamp_iso` columns make the date explicit for long unattended runs and for logs copied later from an SD card. `time_ms` remains useful as a session-relative timeline because it starts at zero and pauses/resumes with logging.

4.3 Which CSV Should I Use?

Use the central collector CSV when you want a complete packet archive from multiple Pis and microphones in one place. It preserves all OSC topics and sender information.

Use the Pi-local analysis CSV when you want a regular time-grid table for one microphone, with VAD, prosody, and emotion values already aligned into rows.

For a live session, it is reasonable to run both: central collection for raw multi-device capture, and Pi-local logging for per-microphone analysis tables.

5. Retrieving Pi-Local Logs

There are two normal ways to retrieve Pi-local CSV logs after a run.

5.1 Network Copy

If the central computer can SSH into the Pis, use `gather_logs.sh`:

```
./gather_logs.sh output/session_001 pi1.local pi2.local
```

If the repository lives at a different path on the Pi, pass the remote output path explicitly:

```
./gather_logs.sh --remote-path SPEECH_RECORD_ANALYSIS/output/ output/session_001 pi1.local
```

5.2 Physical SD-Card Copy

For long multi-day installations, it may be simpler to collect the Pis physically and copy the logs from the SD cards afterward.

1. Shut down the Pi cleanly if possible.
2. Remove the SD card and mount it on another computer.
3. Find the Pi user's home folder on the card.
4. Copy `SPEECH_RECORD_ANALYSIS/output/` to the central archive folder.

Because the Pi-local CSV files contain `session_start_unix_ms`, `session_start_iso`, `timestamp_unix_ms`, and `timestamp_iso`, the files still carry full date/time information even if they are copied days later.

6. Timing, Sampling, And Model Rates

The processing stages do not all run at the same rate.

Stage	Default timing	Notes
Audio capture	Native device rate, resampled to 16 kHz	The input device may run at 44.1 or 48 kHz; the callback resamples to `sample_rate`.
VAD	`vad_interval: 0.25`, `vad_grid_hz: 100`	Silero VAD inspects overlapping audio windows and writes a 100 Hz speech/silence timeline.
openSMILE prosody	`opensmile_interval: 0.5`	openSMILE low-level descriptors are frame-level features with roughly 10 ms hop timing.
Emotion	`emo_window: 2`, `emo_hop: 0.5`	emotion2vec reads a sliding 2 s window every 0.5 s when enabled.
Pi-local CSV logger	`log_interval: 0.25`	Writes the latest known VAD/prosody/emotion values at 4 Hz by default.
OSC stream	logger-driven	Sends VAD, selected prosody features, and emotion scores on the logger tick.
Central collector	packet-driven	Records every incoming OSC packet at the central computer receive time.

Because the models run independently, a CSV row may contain a new VAD value, the latest prosody frame, and an emotion result computed from an earlier 2 s window. This is expected. The row is a synchronized reporting grid, not proof that every model produced a new result at that exact millisecond.

7. Data Values And Ranges

7.1 VAD

`vad` is a gate, not a probability, in the CSV output:

Value	Meaning
`-1`	VAD is disabled or no VAD data exists yet. Downstream stages treat the gate as open.
`0`	VAD is active and the current interval is silence/non-speech.
`1`	VAD is active and the current interval contains speech.

Silero VAD itself uses `vad_threshold` internally. The default in the example config is:

```
vad_threshold: 0.3
```

7.2 Prosody And openSMILE

The live OSC/browser view streams a small subset of openSMILE low-level descriptors:

OSC / CSV key	Approximate display range	Meaning
`F0semitoneFrom27.5Hz_sma3nz`	0 to 50 semitones	Pitch estimate. Zeros/invalid unvoiced frames are treated as gaps.

OSC / CSV key	Approximate display range	Meaning
`Loudness_sma3`	0 to 2.5	Perceptual loudness descriptor.
`jitterLocal_sma3nz`	0 to 0.35	Cycle-to-cycle voice perturbation.
`shimmerLocaldB_sma3nz`	0 to 30 dB	Amplitude perturbation.
`HNRdBACF_sma3nz`	0 to 15 dB in the browser display	Harmonics-to-noise ratio.

The project uses openSMILE eGeMAPSv02 features. For background, see the openSMILE Python package documentation and the eGeMAPS feature set:

- https://audeering.github.io/opensmile-python/
- https://audeering.github.io/opensmile-python/api/opensmile.FeatureSet.html

The older `src/track_writer.py` helper documents an utterance-level 88-column eGeMAPSv02 CSV shape, but the current `strip_monitor.py` runtime writes the live low-level descriptor subset listed above.

7.3 Emotion

Emotion scores come from emotion2vec through FunASR. The CSV keeps a top label plus one score per class:

```
angry, disgusted, fearful, happy, neutral, other, sad, surprised, unknown
```

Treat these scores as model outputs useful for comparison over time, not calibrated psychological measurements. `emo_confidence` is the score of the current top label. The model reads a sliding audio window, so rapid emotion changes will lag behind the microphone signal by design.

7.4 Self Telemetry

When OSC streaming is active, each Pi process also emits:

```
/dev/<device_id>/stats/self <rss_mb> <cpu_pct> <temp_c> <n_threads>
/dev/<device_id>/stats/rate <address> <hz>
```

The central collector records these messages too. They help diagnose whether two mic processes are overloading a Pi.

8. Runtime Switches

The browser receiver and OSC control commands can switch stages on and off while the Pi process keeps running.

Control	Effect
OSC on/off	Starts or pauses outgoing OSC telemetry. The Pi process continues analyzing locally.
Log start/pause/resume/stop	Opens, pauses, resumes, or closes the Pi-local CSV file.
VAD on/off	Enables or disables Silero VAD gating.

Control	Effect
PROS on/off	Enables or disables openSMILE prosody extraction.
EMO on/off	Enables or disables emotion inference if the model was loaded at startup.

Important distinction: `emotion_active` can be toggled at runtime only if `emotion_load: true` was used at process startup. If `emotion_load: false`, the model is not loaded into memory and emotion cannot be turned on later in that same process.

9. Why VAD Matters

VAD should normally be active during real sessions.

When VAD is active, it gates the analysis so silence and background noise are not treated as meaningful speech. Prosody features that depend on voicing, such as pitch, jitter, shimmer, and HNR, are blanked/gapped during silence. Emotion inference also checks the voiced fraction of its analysis window before running.

When VAD is off, the system treats the gate as open. This is useful for debugging because it forces data to flow, but it also means the prosody and emotion stages may process room noise, handling sounds, silence, or electrical noise. In other words, without VAD active, the pipeline can try to interpret garbage as speech.

Use VAD off only when you deliberately want an ungated diagnostic stream.

10. Extending The Collected Features

More openSMILE low-level descriptors can be exposed if needed. The current subset is a deliberate live-display/logging choice, not a hard limit of openSMILE.

To add another live feature:

1. Find the exact openSMILE low-level descriptor column name.
2. Add it to the `FEATURES` list in `strip_monitor.py`. That controls which low-level descriptor columns are read from openSMILE, written into the Pi-local CSV, and emitted over OSC.
3. If the feature should appear in the browser GUI, add a matching entry to the `channels` object in `receiver/sketch.js`. That controls display label, color, and plot range.

The central collector does not need a schema change because it records arbitrary OSC addresses in long format.